

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24

Representing the SKI Combinator Calculus in the Nock Instruction Set Architecture

N. E. Davis `~lagrev-nocfep`
ZeroAttest, Groundwire Foundation

Abstract

The Nock ISA opcodes 2, 1, and 0 have been said to correspond to the combinators S, K, and I in the SKI combinator calculus. We give a four-rule compiler from arbitrary SKI terms to Nock formulas using only opcodes 0, 1, and 2, prove it correct under a fixed application convention, and exercise it on the usual worked examples: the identity laws, Church booleans, the B, C, and W combinators, and Church numerals performing real arithmetic. The compiled output is verified against an independent Nock 4K interpreter. The construction is a constructive proof that the Nock fragment of 0, 1, 2 opcodes is Turing-complete. This fragment lacks certain properties, marking a boundary beyond which the fragment is genuinely insufficient and additional tools are required.

Contents

1	Introduction	2
2	Preliminaries	4

25	3	Combinator Formulas	5
26	3.1	I identity	5
27	3.2	K constant	5
28	3.3	S substitution	5
29	3.4	B composition	6
30	3.5	C transposition	6
31	3.6	W duplicate	7
32	3.7	Commentary	7
33	4	Compilation of the Homomorphism	8
34	5	Worked Examples	9
35	5.1	Identity laws	9
36	5.2	Church booleans	12
37	5.3	The B, C, W combinators	12
38	5.4	Church numerals and arithmetic	13
39	6	Validation	16
40	7	Cost	16
41	8	Limitations	17
42	9	Conditionals	18
43	10	Conclusion	19

1 Introduction

The Nock instruction set architecture is a combinator calculus sufficient to represent all computable functions. It has been a longstanding temptation to draw a direct analogy from the Nock ISA to the SKI combinator calculus. This analogy claims that the Nock opcodes 2, 1, and 0 correspond to the combinators S, K, and I respectively. The analogy is usually stated informally, e.g. “[there are] some subtle differences to Nock’s expression of S as opcode 2 that we will elide as being fundamentally similar, but perhaps worthy of its own monograph” (~lagrev-nocfep and ~sorreg-namtyv, 2025). We here deliver

55 upon that promise and convert an Urbit legend into a concrete
56 demonstration.

57 Combinator calculi build all computation from a handful
58 of variable-free operators; the SKI system does it with three
59 (Curry and Feys, 1958), which is neither unique nor minimal
60 but balances convenience with economy.¹ Put briefly, the S
61 combinator (“substitute”) takes three arguments and applies
62 the first to the third, then applies the second to the third, and
63 finally applies the result of the first application to the result
64 of the second. The K combinator (“k’onstant”) takes two ar-
65 guments and returns the first, discarding the second. The I
66 combinator (“identity”) takes one argument and returns it un-
67 changed. The SKI system is Turing-complete, meaning that
68 any computable function can be expressed as a combination
69 of these three operators.

70 $S\ x\ y\ z = x\ z\ (y\ z)$

71 $K\ x\ y = x$

72 $I\ x = x$

73 The Nock ISA is also Turing-complete, and it is natural to
74 ask whether the SKI combinators can be represented in Nock.
75 The answer is yes, but some details are subtle. The analogy
76 between Nock opcodes and SKI combinators is not a simple
77 one-to-one compilation procedure, but a homomorphism that
78 preserves the structure of the computation.

79 A straightforward reading of Nock opcode 0 yielding I is
80 uncontroversial, as [0 1] trivially returns identity. The read-
81 ing of opcode 1 yielding K is also straightforward, as [1 x]
82 returns a constant function that discards its argument (sub-
83 ject). The reading of opcode 2 yielding S is more subtle, as it
84 requires a fixed convention for how the compiled combinator
85 receives its arguments. The “subtle differences” of *~lagrev-*
86 *nocfep* and *~sorreg-namtyv*’s elision matter more than they
87 appear. The opcode-to-combinator analogy is not a compila-
88 tion procedure. SKI terms are curried and higher-order: com-
89 binators are routinely partially applied, and a combinator may
90 be passed as an argument to another combinator. A symbol-
91 for-symbol substitution—writing 2 for S, 1 for K, and 0 for I,

¹See particularly Smullyan (1994) for more on combinatory logic.

with appropriate transposition of arguments into subject and formula—produces nothing runnable the moment S has fewer than three arguments, which is almost always. What is needed is a *homomorphism*: a translation under which Nock application of compiled terms mirrors SKI reduction, with a fixed convention for how a compiled combinator receives its arguments.

We supply such a homomorphism as four rules with three small Nock formulas yielding the SKI combinators. The remainder of this article consists of worked examples, mechanical validation, a cost analysis, and an accounting of the limitations.

2 Preliminaries

We assume Nock $4K$. Evaluation is the function $\star[\text{subject formula}]$. We require only three opcodes from the Nock ISA:

```

:: slot: fetch the noun at tree address b
 $\star[a\ 0\ b] \rightarrow /[b\ a]$ 

:: quote: return b unchanged
 $\star[a\ 1\ b] \rightarrow b$ 

:: eval: compute a subject and a formula, then run
 $\star[a\ 2\ b\ c] \rightarrow \star[\star[a\ b]\ \star[a\ c]]$ 

```

plus the structural distribution (autocons) rule, which fires whenever a formula's head is itself a cell:

```

 $\star[a\ [b\ c]\ d] \rightarrow [\star[a\ b\ c]\ \star[a\ d]]$ 

```

Slot address 1 denotes the whole subject, so $\star[a\ 0\ 1] = a$.

A *value* is a Nock formula that consumes its argument as the *subject*. To apply a value v to an argument x , we evaluate

```

 $\text{apply}(v, x) == \star[x\ v]$ 

```

This operative convention inverts the lambda-calculus argument order, because in Nock the function is the formula and the argument is the subject.

124 3 Combinator Formulas

125 Under the convention of Section 2, each SKI combinator is a
 126 closed Nock formula. We give each, then verify its defining
 127 law by reduction.

128 3.1 I identity

129 $I\ x = x$ requires $\star[x\ I] = x$, which is slot 1:

130 $I = [0\ 1]$

131 3.2 K constant

132 Applying K to x must yield a value that ignores its next argu-
 133 ment and returns its value. Thus $K\ x\ y = x$ requires $\star[x\ K]$
 134 $= [1\ x]$, which is built from subject x by autocons:

135 $K = [[1\ 1]\ [0\ 1]]$

136 For example,

137 $\star[x\ K] = [\star[x\ [1\ 1]]\ \star[x\ [0\ 1]]] = [1\ x]$

138 Then for any y , $\star[y\ [1\ x]] = x$, so $K\ x\ y = x$ holds.

139 3.3 S substitution

140 Substitution states $S\ x\ y\ z = (x\ z)(y\ z)$. The combinator
 141 accumulates x , then y , then fires on z . We construct it bottom-
 142 up.

143 When the fully-applied $S\ x\ y$ finally receives z as subject,
 144 it must produce $(x\ z)(y\ z)$. Under the convention, $x\ z =$
 145 $\star[z\ x]$ and $y\ z = \star[z\ y]$, and the outer application is $\star[$
 146 $\star[z\ y]\ \star[z\ x]]$. That is precisely opcode z with subject z ,
 147 formula-b equal to y , formula-c equal to x :

148 $S\ x\ y = [2\ y\ x]$

149 $\therefore \star[z\ [2\ y\ x]] = \star[\star[z\ y]\ \star[z\ x]] = (x\ z)(y\ z)$

150 Working back one step, applying $S\ x$ to y (subject y , constant
 151 x) must build the noun $[2\ y\ x]$:

152 $S\ x = [[1\ 2]\ [0\ 1]\ [1\ x]]$

153 We check that $\star[y\ S\ x] = [2\ y\ x]$, since $\star[y\ [1\ 2]] =$
 154 2 , $\star[y\ [0\ 1]] = y$, and $\star[y\ [1\ x]] = x$. Working back the
 155 final step, applying S to x (subject x) must build $S\ x$; the first
 156 two parts are constants and the third, $[1\ x]$, is built from x by
 157 the same autocons trick used for K :

158 $S = [[1\ [1\ 2]]\ [1\ [0\ 1]]\ [[1\ 1]\ [0\ 1]]]$

159 We check the evaluation: $\star[x\ S] = [[1\ 2]\ [0\ 1]\ [1\ x]]$
 160 $= S\ x$.

161 3.4 B composition

162 Any other combinator can be reached by elaborating its lambda
 163 definition to SKI and compiling, but the result is large. A named
 164 combinator is better derived directly, by the same method as
 165 the previous subsection: it becomes one builder layer per argu-
 166 ment. The innermost layer is the opcode 2 application skeleton
 167 that fires on the final argument; each enclosing layer is an au-
 168 tocons that captures one argument and embeds it. We here de-
 169 rive B , C , and W ; a similar methodology could be used for much
 170 of Smullyan's aviary (Smullyan, 1994).

171 For $B\ f\ g\ x = f\ (g\ x)$, the firing step computes f ap-
 172 plied to $g\ x$, i.e. $\star[\star[x\ g]\ f]$, which is opcode 2 with g in
 173 the formula-b slot and f quoted in the formula-c slot; captur-
 174 ing g then f gives:

175 $::\ \star[x\ B\ f\ g] = \star[\star[x\ g]\ f] = f\ (g\ x)$

176 $B\ f\ g = [2\ g\ [1\ f]]$

177 $::\ \star[g\ B\ f] = [2\ g\ [1\ f]]$

178 $B\ f = [[1\ 2]\ [0\ 1]\ [1\ [1\ f]]]$

179 $::$

180 $B = [[1\ [1\ 2]]\ [1\ [0\ 1]]\ [[1\ 1]\ [[1\ 1]\ [0\ 1]]]]$

181 3.5 C transposition

182 The same procedure yields C (flip):

183 $::\ f\ x\ y \rightarrow f\ y\ x$

184 $C = [[1\ 1\ 2]\ [1\ [1\ 1]\ 0\ 1]\ [1\ 1]\ 0\ 1]$

185 3.6 W duplicate

186 Likewise, W (duplicate):

```
187 :: f x    -> f x x
188 W = [[1 2] [1 0 1] 0 1]
```

189 3.7 Commentary

190 The one subtlety is how a captured argument is embedded,
 191 which is dictated by how the combinator uses that argument. If
 192 the argument is applied to a computed value (as B's f is applied
 193 to g x), it occupies a formula slot and is embedded quoted, [1
 194 f]. If it is applied to the current subject (as W's f is applied to
 195 x) it must pass through as the live formula, embedded by slot,
 196 [0 1]. Quote where the argument is data; slot where the argu-
 197 ment is the active function.

198 Notably, the value forms of B, C, and W use only Nock op-
 199 codes 0 and 1. The opcode 2 appears only in the formula they
 200 assemble at application time ([2 g [1 f]]), carried as quoted
 201 data. The combinators are pure construction; the application
 202 machinery is the structure they emit. Among the basis only S
 203 carries a 2 in its own body, because it performs two applica-
 204 tions at once.

205 The size payoff over bracket abstraction is large (cell counts;
 206 see Section 7 for method):

	direct	via SKI	ratio
207 B	11	208	19×
C	11	208	19×
W	6	130	22×

208 The two forms are extensionally identical on every input
 209 tested. For any combinator known by name, direct derivation
 210 is preferred and the abstraction tax is avoided entirely; bracket
 211 abstraction remains the general fallback for arbitrary lambda
 212 terms.

213 4 Compilation of the Homomorphism

214 Let $[[t]]$ denote a *closed* Nock formula whose product is the
215 value of the SKI term t . The compiler consists of four rules:

216 $[[S]] = [1\ S]$
217 $[[K]] = [1\ K]$
218 $[[I]] = [1\ I]$
219 $[[A\ B]] = [2\ [[B]]\ [[A]]]$

220 with S, K, I the formulas of Section 3. The combinators are
221 *quoted* so that $[[\cdot]]$ always denotes a value-producing com-
222 putation; application is opcode 2 applied to the two compiled
223 operands. $[[t]]$ produces the value of t ; it is closed, so the
224 naïve approach of $*[a\ [[t]]]$ yields $\text{val}(t)$ for any subject
225 a and is not yet an application. To apply t to Nock-noun argu-
226 ments $a[0] \dots a[n]$, fold them onto the value, each quoted,
227 by opcode 2:

228 $\text{run}(t; a[1] \dots a[n]) = *[0\ F[n]]$

229 where $F[0] = [[t]]$, $F[i] = [2\ [1\ a[i]]\ F[i-1]]$. Each
230 layer $*[\cdot\ [2\ [1\ a[i]]\ F]]$ reduces to $*[a[i]\ \text{val}]$, the value
231 applied to $a[i]$ as subject. Throughout the remainder of this
232 paper, we will write $t\ a[1] \dots a[n] = r$ as shorthand for
233 $\text{run}(t; a[1] \dots a[n]) = r$. The arguments are Nock nouns,
234 not SKI terms: atoms in the identity and boolean laws, real for-
235 mulas such as $+/\text{INC}$ in the arithmetic examples. A tap must not
236 fire until its formula is a concrete noun.

237 Why quote, and why this order? Evaluating $[[A\ B]] =$
238 $[2\ [[B]]\ [[A]]]$ against any subject s gives $*[*[s\ [[B]]]$
239 $*[s\ [[A]]]]$. Because $[[B]]$ and $[[A]]$ are closed, they ig-
240 nore s and reduce to the values $\text{val}(B)$ and $\text{val}(A)$. The re-
241 sult is $*[\text{val}(B)\ \text{val}(A)]$ —apply value $\text{val}(A)$ to argument
242 $\text{val}(B)$, i.e. A applied to B . The operands are reduced to val-
243 ues *before* application, which is what makes nested applica-
244 tions compose correctly; quoting a sub-application rather than
245 evaluating it produces an “off-by-one error” (a residual K where
246 the value was expected).

247 Correctness is demonstrated practically by induction
248 on term structure. For application, the rule computes

249 `apply(val(A), val(B))`, and by the induction hypothesis
 250 `val(A), val(B)` are the values of `A, B`; by §2 this is the value
 251 of `A B`. The translation is therefore a homomorphism from SKI
 252 application to Nock evaluation, under the call-by-value strat-
 253 egy forced by opcode 2's eager semantics (see Section 8).

254 5 Worked Examples

255 All terms below are written in lambda form for legibility, elab-
 256 orated to SKI by standard bracket abstraction (Kiselyov, 2018),
 257 then compiled by the Nock SKI compiler and run. The results
 258 are the products actually computed.

259 A script which resolves the SKI terms to Nock formulas,
 260 runs them in a Nock 4K interpreter, and checks the results is
 261 available at `sigilante/skitrace`.

262 5.1 Identity laws

263 From canonical SKI, we anticipate the following identity laws
 264 to hold true:

265	<code>I 42</code>	<code>= 42</code>	
266	<code>S K K 42</code>	<code>= 42</code>	<code>:: S K K → I</code>
267	<code>S K S 42</code>	<code>= 42</code>	<code>:: S K * → I</code>

268 The evaluation of `S K K 42`, worked in Nock SKI, is shown in
 269 Listings 1, 2, and 3. The process is mechanical and discursive.
 270 The evaluation of `S K S 42` is similar and left as a proverbial
 271 exercise to the reader.

Listing 1: Compilation expansion of `S K K 42`.

272	<code>*[42 [[S K K]]]</code>	
273	<code>*[42 2 [[K]] [[S K]]]</code>	
274		<code>[[A B]] = [2 [[B]] [[A]]]</code>
275	<code>*[42 2 [1 K] [[S K]]]</code>	
276		<code>[[K]] = [1 K]</code>
277	<code>*[42 2 [1 K] 2 [[K]] [[S]]]</code>	
278		<code>[[A B]] = [2 [[B]] [[A]]]</code>
279	<code>*[42 2 [1 K] 2 [1 K] [[S]]]</code>	
280		<code>[[K]] = [1 K]</code>

```

281 * [42 2 [1 K] 2 [1 K] 1 S]
282           [[S]] = [1 S]
283 * [42 2 [1 [1 1] 0 1] 2 [1 [1 1] 0 1] 1 [1 1 2] [1 0 1] [1
284   1] 0 1]
285           substitute S,K,I formulas

```

Listing 2: Evaluation of S K K 42.

```

286 * [42 2 [1 [1 1] 0 1] 2 [1 [1 1] 0 1] 1 [1 1 2] [1 0 1] [1
287   1] 0 1]
288           Nock 2 * [a 2 b c] = * [* [a b] * [a c]]
289 * [* [42 1 [1 1] 0 1] * [42 2 [1 [1 1] 0 1] 1 [1 1 2] [1 0
290   1] [1 1] 0 1]]
291           Nock 1 * [a 1 b] = b
292 * [[[1 1] 0 1] * [42 2 [1 [1 1] 0 1] 1 [1 1 2] [1 0 1] [1
293   1] 0 1]]]
294           Nock 2 * [a 2 b c] = * [* [a b] * [a c]]
295 * [[[1 1] 0 1] * [* [42 1 [1 1] 0 1] * [42 1 [1 1 2] [1 0 1]
296   [1 1] 0 1]]]
297           Nock 1 * [a 1 b] = b
298 * [[[1 1] 0 1] * [[[1 1] 0 1] * [42 1 [1 1 2] [1 0 1] [1 1]
299   0 1]]]
300           Nock 1 * [a 1 b] = b
301 * [[[1 1] 0 1] * [[[1 1] 0 1] [1 1 2] [1 0 1] [1 1] 0 1]]
302           cons * [a [b c] d] = [* [a b c] * [a d]]
303 * [[[1 1] 0 1] * [[[1 1] 0 1] 1 1 2] * [[[1 1] 0 1] [1 0 1]
304   [1 1] 0 1]]]
305           Nock 1 * [a 1 b] = b
306 * [[[1 1] 0 1] [1 2] * [[[1 1] 0 1] [1 0 1] [1 1] 0 1]]
307           cons * [a [b c] d] = [* [a b c] * [a d]]
308 * [[[1 1] 0 1] [1 2] * [[[1 1] 0 1] 1 0 1] * [[[1 1] 0 1]
309   [1 1] 0 1]]]
310           Nock 1 * [a 1 b] = b
311 * [[[1 1] 0 1] [1 2] [0 1] * [[[1 1] 0 1] [1 1] 0 1]]
312           cons * [a [b c] d] = [* [a b c] * [a d]]
313 * [[[1 1] 0 1] [1 2] [0 1] * [[[1 1] 0 1] 1 1] * [[[1 1] 0
314   1] 0 1]]]
315           Nock 1 * [a 1 b] = b
316 * [[[1 1] 0 1] [1 2] [0 1] 1 * [[[1 1] 0 1] 0 1]]
317           Nock 0 * [a 0 b] = /[b a]
318 * [[[1 1] 0 1] [1 2] [0 1] 1 /[1 [[1 1] 0 1]]]
319           /[1 a] = a
320 * [[[1 1] 0 1] [1 2] [0 1] 1 [1 1] 0 1]

```

```

321             cons  *[a [b c] d] = [*[a b c] *[a d]]
322  [*[[[1 1] 0 1] 1 2] *[[[1 1] 0 1] [0 1] 1 [1 1] 0 1]]
323             Nock 1  *[a 1 b] = b
324  [2 *[[[1 1] 0 1] [0 1] 1 [1 1] 0 1]]
325             cons  *[a [b c] d] = [*[a b c] *[a d]]
326  [2 *[[[1 1] 0 1] 0 1] *[[[1 1] 0 1] 1 [1 1] 0 1]]
327             Nock 0  *[a 0 b] = /[b a]
328  [2 /[1 [[1 1] 0 1]] *[[[1 1] 0 1] 1 [1 1] 0 1]]
329             /[1 a] = a
330  [2 [[1 1] 0 1] *[[[1 1] 0 1] 1 [1 1] 0 1]]
331             Nock 1  *[a 1 b] = b
332  [2 [[1 1] 0 1] [1 1] 0 1]
333  (= [2 K K], the identity combinator's value)

```

Listing 3: Application of S K K 42.

```

334  # applying val(S K K) to 42  ( *[42 val(S K K)] )
335  val(S K K) = [2 [[1 1] 0 1] [1 1] 0 1]
336
337  # evaluation of *[42 [2 [[1 1] 0 1] [1 1] 0 1]]
338  *[42 2 [[1 1] 0 1] [1 1] 0 1]
339             Nock 2  *[a 2 b c] = *[*[a b] *[a c]]
340  *[*[42 [1 1] 0 1] *[42 [1 1] 0 1]]
341             cons  *[a [b c] d] = [*[a b c] *[a d]]
342  *[[*[42 1 1] *[42 0 1]] *[42 [1 1] 0 1]]
343             Nock 1  *[a 1 b] = b
344  *[[1 *[42 0 1]] *[42 [1 1] 0 1]]
345             Nock 0  *[a 0 b] = /[b a]
346  *[[1 /[1 42]] *[42 [1 1] 0 1]]
347             /[1 a] = a
348  *[[1 42] *[42 [1 1] 0 1]]
349             cons  *[a [b c] d] = [*[a b c] *[a d]]
350  *[[1 42] *[42 1 1] *[42 0 1]]
351             Nock 1  *[a 1 b] = b
352  *[[1 42] 1 *[42 0 1]]
353             Nock 0  *[a 0 b] = /[b a]
354  *[[1 42] 1 /[1 42]]
355             /[1 a] = a
356  *[[1 42] 1 42]
357             Nock 1  *[a 1 b] = b
358  42

```

5.2 Church booleans

Conventional Nock utilizes atoms, but SKI combinators are pure construction; we therefore must utilize Church-style booleans and numerals within the particular three-opcode discipline we follow here. Thus while this section is demonstrative of SKI in Nock, it evades the usual Nock idioms and in particular the economy afforded by the introduction of opcodes 3, 4, and 5.

A Church boolean is a two-argument function that selects one of its arguments. Because the combinators are pure construction, the truth values are instead represented as combinator sequences. With $\text{TRUE} = K$ and $\text{FALSE} = K I$:

```

TRUE p q
  = K p q
  = p
FALSE p q
  = K I p q
  = I q
  = q
TRUE 7 8 = 7
FALSE 7 8 = 8

```

These are the canonical encodings from SKI praxis; here we probe with atom arguments because neither combinator applies its arguments as functions.

5.3 The B, C, W combinators

Elaborated from their defining lambda terms:

```

B = λ f g x. f (g x)    compose
C = λ f x y. f y x      transpose
W = λ f x. f x x        duplicate

```

Driving B with a real Nock increment $\text{inc} = [4\ 0\ 1]$ as payload, and C, W with atoms:

```

B inc inc 7 = 9      :: inc (inc 7)
C K 7 8     = 8      :: K 8 7 = 8
W K 7       = 7      :: K 7 7 = 7

```

395 These exercise three-variable abstraction and argument dupli-
 396 cation. Note that the compiled SKI scaffolding remains pure 0,
 397 1, 2; the only 4 in B's run is the external increment supplied as
 398 data.²

399 5.4 Church numerals and arithmetic

400 A Church numeral `church n` applies its first argument `n` times
 401 to its second. Numerals are built and incremented entirely
 402 within the fragment.³ Examples of Church numerals and their
 403 Nock SKI representations are given in Table 1. At this point,
 404 we will revert in part to the lambda calculus notation for clar-
 405 ity, but the underlying Nock SKI formulas are always available
 406 and are used in the actual evaluation. The successor function
 407 is defined by the usual combinator formula:

```
408 church n =  $\lambda\lambda f.x. f^n x$ 
409 succ =  $\lambda\lambda n.f.x. f (n f x)$ 
410
411 succ (church 0) = church 1
412 succ (church 1) = church 2
```

413 In explicit Nock SKI form, the Church numeral successor func-
 414 tion becomes:

```
415 succ = [[1 1 2] [0 1] 1 [1 1] 0 1]
416
417 church 2 = [[1 2] [[1 2] [1 0 1] [1 1] 0 1]
418               [1 1] 0 1]
419 [church 2] = 2
420 succ (church 2) = [[1 2] [[1 2] [[1 2] [1 0 1] [1 1] 0 1]
421                           [1 1] 0 1] [1 1] 0 1]
422 [succ (church 2)] = 3
```

²We here draw a sharp distinction between opcode 4 as `inc` = [4 0 1] which operates on Nock atoms, and `succ` which operates on Church numerals. The two are distinct: `succ` maps a numeral-value to a numeral-value in 0,1,2, while `inc` maps an atom to an atom via opcode 4.

³A Church numeral is a value (formula), not an atom. To read it out as a Nock atom we decode it: apply it to the atom successor `inc` = [4 0 1] and the atom 0, writing `[n]= n inc 0`: `[church 0]= 0` ; `[SUCC*5 (church 0)]= 5` ; `[SUCC (church 2)]= 3`. Decoding is the only step that uses opcode 4, and it is readout, not arithmetic.

Arithmetic over the Church numerals is defined by the usual combinator formulas:

```

423  :: in Church numerals
424
425  plus = λλλλm.n.f.x. m f (n f x)
426
427  mult = λλλλm.n.f. m (n f)
428
429  :: alternatively
430  plus = λm n. m succ n
431  mult = λm n. m (plus n) (church 0)
432
433  :: in Nock decoding:
434  plus = [[1 1 1 2] [1 0 1] [1 1 1 1] [1 1] 0 1]
435  mult = [[1 1 2] [1 0 1] [1 1 1] [1 1] 0 1]
436
437  succ (church 2) = church 3
438
439  :: in Church numerals:
440  church 0 succ 0 = 0
441  church 3 succ 0 = [[1 2] [[1 2] [[1 2] 0 [1 1] 0 1] [1
442    1] 0 1] [1 1] 0 1]
443  church 5 succ 0 = 5
444  (plus 2 3) succ 0 = 5
445  (mult 3 4) succ 0 = 12
446
447  :: in Nock decoding:
448  [church 0] = 0
449  [church 3] = 3
450  [church 5] = 5
451  [plus 2 3] = 5
452  [mult 3 4] = 12

```

Table 1: Church numerals and their Nock SKI representations. The numeral `church n` is a value (formula) that applies its first argument `n` times to its second. The decoding `[n] = n inc 0` produces the Nock atom corresponding to the numeral.

<code>n</code>	<code>[n]</code>	<code>church n</code>
0	0	<code>[1 0 1]</code>
1	1	<code>[[1 2] [1 0 1] [1 1] 0 1]</code>
2	2	<code>[[1 2] [[1 2] [1 0 1] [1 1] 0 1] [1 1] 0 1]</code>
3	3	<code>[[1 2] [[1 2] [[1 2] [1 0 1] [1 1] 0 1] [1 1] 0 1] [1 1] 0 1]</code>
4	4	<code>[[1 2] [[1 2] [[1 2] [[1 2] [1 0 1] [1 1] 0 1] [1 1] 0 1] [1 1] 0 1] [1 1] 0 1]</code>

Table 2: Compiled Nock cell counts for various SKI terms.

Term	SKI leaves	Nock cells
I	1	2
S K K	3	22
W	16	130
B	25	208
C	25	208
church 2	76	640
church 5	187	1588
church 10	372	3168

6 Validation

Any valid Nock ISA interpreter should correctly evaluate the examples above after the SKI compiler stage has been applied. We utilized both a simple interpreter with only opcodes 0, 1, and 2,⁴ and the `pinochle` interpreter with the full Nock ISA.⁵ Results were identical (and correct) in both cases. The examples are not exhaustive, but they exercise the combinators and the compiler in a variety of ways, including nested applications, argument duplication, and Church-style numerals and booleans.

7 Cost

The transformation is faithful but not frugal. Measuring SKI leaf count against compiled Nock cell count shows the extreme verbosity of the latter. Table 2 shows the compiled size of several terms, including the Church numerals and the named combinators.

Compiled Nock runs roughly $8.5\times$ the SKI leaf count (a constant per combinator plus the opcode 2 application wrapper) and the SKI itself is already exponential in the worst case under naive bracket abstraction. The natural number ten com-

⁴`sigilante/skitrace`

⁵`sigilante/pinochle`.

473 piles to a 3168-cell formula as a Church numeral. These figures
 474 are static, before reduction; at runtime ‘S’ duplicates its argu-
 475 ment into both branches with no sharing, so evaluation cost
 476 compounds on representation cost.

477 Named combinators escape this blow-up before the com-
 478 piler stage: derived directly rather than routed through bracket
 479 abstraction, B, C, and W are roughly twenty times smaller. The
 480 cost above is the price of the *general* compiler on arbitrary
 481 terms, not a property of the targets themselves.

482 8 Limitations

483 Three properties bound the construction. Each is a real bound-
 484 ary, not a deficiency to be patched away within the fragment.

485 1. **Call-by-value.** Opcode 2 is eager: $\star[a\ 2\ b\ c]$ fully re-
 486 duces $\star[a\ b]$ and $\star[a\ c]$ before applying. The compi-
 487 lation is therefore applicative-order. A term whose nor-
 488 mal form depends on discarding a divergent subterm will
 489 loop. Concretely, with $\mathbb{W} = S\ I\ I$ and $\Omega = \mathbb{W}\ \mathbb{W}$, normal-
 490 order $K\ I\ \Omega$ reduces to I , but the compiled form diverges.
 491 For a compilation target this is usually acceptable; for a
 492 faithful lazy surface it is not, and recovering normal or-
 493 der requires explicit thunking, which reintroduces run-
 494 time branching and pulls in opcodes 3 and 6.

495 2. **No readback in {0, 1, 2}.** Compiled SKI can be exe-
 496 cuted but its normal form cannot be printed without dis-
 497 tinguishing a residual S from a K from an application at
 498 runtime. For Nock, this requires a structural test (opcode
 499 3) and a conditional (opcode 6). The fragment is a faithful
 500 execution target, not a normalizer.

501 A normalizer must additionally return the normal form
 502 in an inspectable form, which means recognizing com-
 503 binators at runtime, which the {0, 1, 2} fragment cannot
 504 do. In other words, when an SKI evaluator completes,
 505 it has a normal form or a tree whose leaves are S , K , I
 506 and whose internal nodes are applications. To print that

normal form (to “read it back”), one walks the tree and at each node decides if it is a leaf or an application, and if a leaf, which combinator?

3. **No sharing.** This is tree reduction. *S* copies its argument with no sharing, so terms with shared redexes blow up exponentially. This is moderately acceptable as an intermediate representation to compile through, but impractical as a runtime for large terms, which want graph reduction.

These can be read as an alternate set of motivations for opcodes beyond 2; as may be surmised, the {0, 1, 2} fragment is Turing-complete but lacks certain practical features.

9 Conditionals

A conditional is an obvious stress test for such a small fragment because in a strict evaluator a conditional that evaluates both branches is useless for its principal job of guarding recursion. In practice, the conditional must be split into two parts.

The Church booleans are already a form of conditional; with `TRUE = K` and `FALSE = K I`, the term `b t e` routes to the chosen branch. The routing itself executes neither branch: *K* reads its two arguments by slot and discards one, never applying them as formulas. If the branches are held as quoted formulas (thunks) and only the survivor is run, the unchosen branch is never evaluated. The whole conditional is one closed formula in the fragment:

```
IF cond tf ef s
    = [2 [1 s] [2 [1 ef] [2 [1 tf] [1 cond]]]]
```

Reading inner to outer: apply `cond` to `tf`, then to `ef` (selection), then run the survivor on subject *s*. It is fifteen cells, pure {0, 1, 2}.

This conditional is genuinely lazy.⁶ The discipline this re-

⁶Setting the unchosen branch to a crashing formula confirms it never runs: `IF TRUE 111 ⊥` returns 111 without crashing, `IF FALSE ⊥ 222` returns 222, and the control case `IF TRUE ⊥ 222` does crash. The chosen branch really executes, so the laziness is not an artifact of dead code.

quires is exact: branches must be kept as quoted formulas rather than compiled as combinator sub-terms. Compile a divergent branch and the eager opcode 2 forces it before selection ever happens (the $\mathbb{K} \mathbb{I} \Omega$ divergence). Thunking the branch keeps it inert until forced; this is similar to stored procedures in conventional Nock using cores with opcode 9.

What the SKI/{0, 1, 2} fragment cannot do effectively is manufacture the boolean. IF above routes on a boolean it is handed; to branch on a property of data (“is this atom zero”, “is this noun a cell”), one must first map a datum to $\mathbb{K}$ or $\mathbb{K} \mathbb{I}$, and that inspection reads an atom’s value or tests cell-ness. Neither is expressible in pure {0, 1, 2}. Selecting on a given boolean is free; deciding what the boolean should be requires the cell test (opcode 3), increment, equality (opcode 5), or negation (built with opcode 6 on opcode 5).

This is precisely the shape of Nock’s own conditional. Opcode 6 is trivially derivable from opcodes 0–5, and what it adds over the selector above is the data-derived predicate: run a formula b to compute a loobean on the subject, map that loobean to an address with (opcode 4) to pick one of two formulas, and run only the chosen one. The combinatory core we have built here is the selection half; the predicate half is a partial motivation for opcodes 3, 4, and 5 as minimal additions to the SKI basis rather than merely conveniences.

10 Conclusion

The SKI construction in Nock ISA is Turing-complete, but it does not have laziness, normal-form readback, or sharing. Each of these properties marks a boundary where the fragment is genuinely insufficient and additional tooling beyond {0, 1, 2} is required.

This article makes concrete the claim that Nock opcodes 0, 1, and 2 are sufficient to represent the SKI combinator calculus and are thus Turing-complete (if insufficient to handle nouns natively). The embedding is a constructive proof that the opcode 0, 1, 2 fragment is Turing-complete: the deferred monograph the *Documentary History* expected is rooted in a hand-

ful of lines of Nock. The compiler is a homomorphism from SKI application to Nock evaluation, under a fixed call-by-value convention. The combinators are pure construction; opcode 2 appears only in the formula they assemble at application time, carried as quoted data. A second corollary locates the fragment’s edge precisely: a conditional’s *selection* is combinatory and lives in 0, 1, 2, but its *decision*—computing a boolean from data—does not, which motivates Nock’s basis as an SKI core plus a minimal inspection and arithmetic kernel. The Nock ISA has been manifestly Turing-complete since its inception, but this exposition makes a new demonstration explicit and constructive.☒

References

- Curry, Haskell B. and Robert Feys (1958). *Combinatory Logic*. Amsterdam: North-Holland Publishing Company.
- Kiselyov, Oleg (2018). “ λ to SKI, Semantically: Declarative Pearl.” In: *Functional and Logic Programming*. Ed. by John P. Gallagher and Martin Sulzmann. Cham: Springer International Publishing, pp. 33–50.
- ~lagrev-nocfep, N. E. Davis and Curtis Yarvin ~sorreg-namtyv (2025). “A Documentary History of the Nock Combinator Calculus.” In: *Urbit Systems Technical Journal* 2.1, pp. 155–190. URL: <https://urbitsystems.tech/article/v02-i01/a-documentary-history-of-the-nock-combinator-calculus>.
- Smullyan, Raymond M. (1994). *To Mock a Mockingbird and Other Logic Puzzles: : Including an Amazing Adventure in Combinatory Logic*. New York: Knopf.